Clara AI Automation Assignment

Transcript-to-Agent Configuration Automation System

Name : L Shravya
Reg No : 22BCE0804

Mail: shravyasetty57@gmail.com(personal)

Shravya.l2022@vitstudent.ac.in **(vit mail)**

GITHUB Repository Link:

https://github.com/ShravyaLagishetty/clara-ai-automation-assignment

Introduction

Artificial intelligence voice agents are increasingly being used by service-based businesses to automate customer interactions. Clara AI provides an intelligent voice assistant platform designed specifically for trade service businesses such as electricians, HVAC companies, plumbing services, and fire protection contractors. These businesses often receive a large number of incoming calls regarding service requests, emergencies, inspections, quotes, and general inquiries.

Before an AI agent can be deployed for a business, the agent must be configured to understand the business workflow. This configuration typically includes information such as the services offered, emergency definitions, routing rules, business hours, and integration constraints. Traditionally, this information is gathered through conversations between Clara's team and the business owner during sales demo calls and onboarding meetings.

However, these conversations exist in the form of transcripts and are not immediately usable for configuring AI systems. Therefore, an automated process is required to transform conversation transcripts into structured configuration files that the Clara AI system can use.

This assignment focuses on designing and implementing an automation pipeline that converts business conversation transcripts into structured AI agent configurations. The system processes two different types of transcripts: demo call transcripts and onboarding call transcripts. Each transcript type contributes different information toward building a complete agent configuration.

The solution is implemented using the workflow automation platform n8n, running locally using Docker. The pipelines extract relevant information from transcripts and generate structured configuration outputs in JSON format.

Problem Statement

The primary challenge addressed in this assignment is the transformation of unstructured conversational transcripts into structured configuration data suitable for AI voice agent deployment.

The system must be able to:

• Process demo call transcripts to understand the business workflow.
• Extract key operational details about the company.
• Generate an initial configuration for the AI agent (version 1).
• Process onboarding call information that contains refined or corrected business details.
• Update the original configuration to create an improved version (version 2).
• Track the changes made between configuration versions.

The automation must be able to handle incomplete or uncertain information by explicitly marking unknown fields. This ensures that missing information can later be confirmed during onboarding.

System Overview

The automation system consists of two independent but connected pipelines. Each pipeline processes a specific type of transcript and produces configuration files for the Clara AI system.

The first pipeline processes demo call transcripts. These transcripts typically contain an exploratory discussion between the Clara team and a potential client. During these conversations, the Clara team learns about the company's services, workflows, and operational requirements. The pipeline extracts this information and generates an initial agent configuration.

The second pipeline processes onboarding call transcripts. These transcripts occur after the business decides to use Clara AI. During onboarding, the business provides additional information such as official contact details, referral partners, and confirmed operational policies. The pipeline updates the initial configuration using this new information.

Both pipelines generate structured outputs in JSON format and store them in a versioned directory structure.

System Architecture

The automation architecture consists of two major processing stages.

The first stage processes demo call transcripts and generates a preliminary agent configuration.

The second stage processes onboarding information and updates the configuration with verified details.

SYSTEM FLOW

Demo Call Transcript
↓
Pipeline A – Demo Call Processing
↓
Extract business information
↓
Generate memo.json
↓
Generate agent_config.json
↓
Store configuration (v1)

Onboarding Call Transcript
↓
Pipeline B – Onboarding Processing
↓
Parse onboarding details
↓
Load previous configuration
↓
Merge updates
↓
Generate updated configuration
↓
Create changelog
↓
Store configuration (v2)

This architecture ensures that configuration updates are handled systematically and that earlier configuration versions remain available for reference.

Tools and Technologies Used

Several technologies were used to build and execute the automation system.

- ⇒ n8n(Self-Hosted)
  The automation workflows were implemented using n8n, an open-source workflow automation platform. n8n allows workflows to be built visually using nodes, making it ideal for integrating data processing tasks.
- ⇒ Docker
  Docker was used to run n8n locally in a containerized environment. This ensures that the automation environment remains consistent and easy to deploy.
- ⇒ JavaScript (n8n Code Nodes)
  Custom JavaScript code was written within n8n code nodes to parse transcript text and extract structured information.
- ⇒ JSON
  JSON was used as the output format for storing structured configuration files. These files include memo.json, agent_config.json, and changelog.json.

⇒ GitHub

GitHub was used to manage version control and submit the completed assignment repository.

⇒ Asana

Asana was used as a task management platform to track assignment progress and organize development steps.

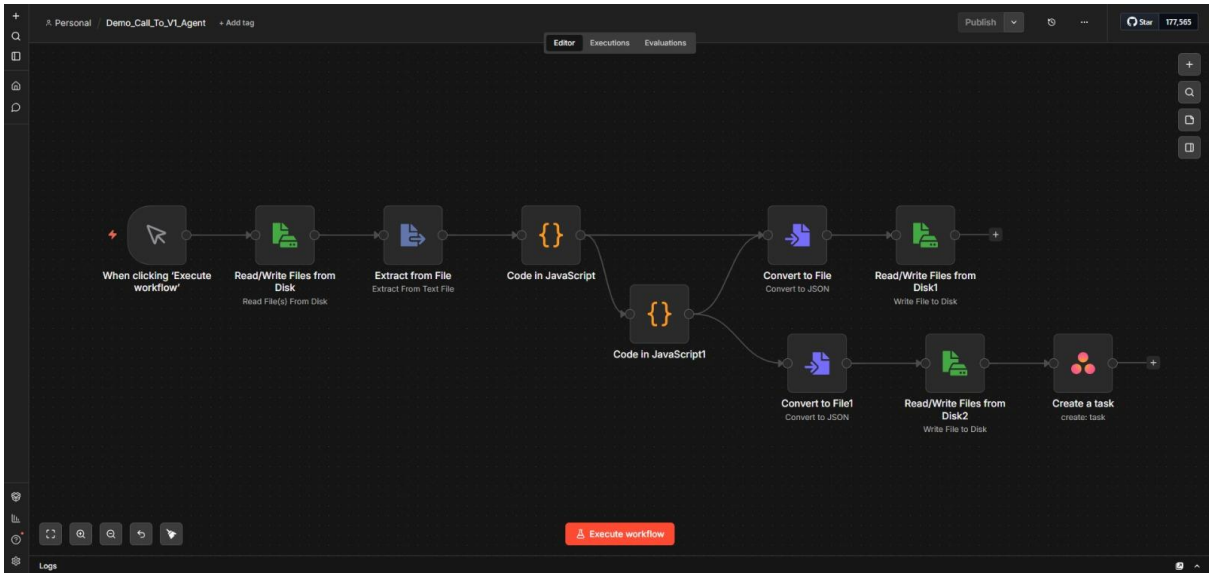
Workflow Implementation

The automation system was implemented using **two separate n8n workflows**, each responsible for processing a different stage of the Clara AI configuration lifecycle.

The workflows were executed locally using **n8n running inside Docker** and used **JavaScript code nodes** to perform transcript parsing and data transformation.

Pipeline A – Demo Call to Initial Agent Configuration

Pipeline A processes the **demo call transcript** and generates the initial AI agent configuration (version 1).



The workflow follows a sequence of nodes that load the transcript, extract information, generate structured JSON files, and store them in the output directory.

Workflow Steps :

Step 1 — Trigger Execution
The workflow begins with the **Manual Trigger node** which starts the workflow execution when the user clicks the "Execute Workflow" button.

Step 2 — Read Transcript File
The **Read/Write Files from Disk node** loads the demo call transcript file from the local filesystem.

Step 3 — Extract Transcript Text
The **Extract from File node** reads the text content of the transcript and passes it to the processing node.

Step 4 — Business Data Extraction
A **JavaScript Code Node** processes the transcript text and extracts key business information using pattern matching and regular expressions.

Extracted information includes:

• Company name
• Services supported
• Emergency definitions
• After-hours routing logic
• Business workflow details

Step 5 — Generate Memo Structure
The extracted information is structured into a **memo JSON object** representing the business configuration.

Step 6 — Generate Agent Configuration
Another **JavaScript Code Node** converts the memo data into a **Retell-compatible AI agent configuration**, defining the behavior of the Clara AI voice assistant.

Step 7 — Convert Data to JSON Files
The structured objects are converted into JSON files using the **Convert to File node**.

Step 8 — Save Configuration Files
The final configuration files are saved to disk using the **Write File node**.

Generated files include:

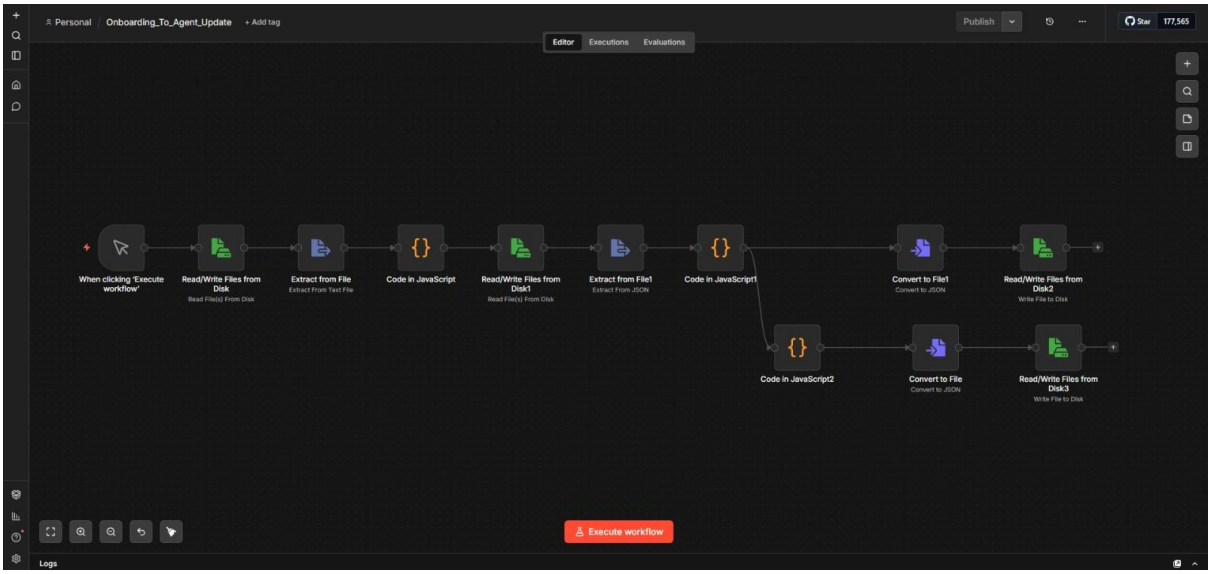
memo.json
agent_config.json

These files are stored under:

outputs/accounts/{account_id}/v1/

This directory represents the **initial version of the configuration**.

Pipeline B – Onboarding Call to Configuration Update



Pipeline B processes the **onboarding call transcript** and updates the previously generated configuration to create **version 2**.

The onboarding call typically includes additional information such as:

• Contact details
• Email addresses
• Referral partners
• Integration requirements

This pipeline merges the new data with the existing configuration.

Workflow Steps :

Step 1 — Workflow Trigger
The pipeline begins with the **Manual Trigger node**.

Step 2 — Load Onboarding Transcript
The onboarding transcript file is loaded using the **Read/Write Files from Disk node**.

Step 3 — Extract Chat Messages
The **Extract from File node** reads the chat transcript text.

Step 4 — Parse Onboarding Information
A **JavaScript Code Node** processes the chat messages and extracts structured information using **regular expressions**.

Extracted data includes:

• Contact name
• Contact email
• Phone number

• Referral partner company
• Partner email

Step 5 — Load Existing Memo
The workflow loads the existing configuration generated during the demo stage using another **Read File node**.

Step 6 — Merge Onboarding Updates
A second **JavaScript Code Node** updates the existing memo structure using the onboarding data.

Updates may include:

• Adding contact information
• Updating routing rules
• Adding integration constraints
• Updating service handling policies

Step 7 — Generate Updated Files
The updated configuration is converted to JSON format.

Step 8 — Save Updated Configuration
The new files are saved under the **v2 directory**, representing the updated configuration version.

Generated files include:

memo.json (updated)
agent_config.json (updated)
changelog.json

Data Storage and Versioning

⇒ The system uses a versioned directory structure to store configuration files.
⇒ Each business account has its own directory under the outputs/accounts folder.
⇒ Within this directory, configurations are organized into version folders.
⇒ Version 1 contains the initial configuration generated from the demo call transcript.
⇒ Version 2 contains the updated configuration generated after processing onboarding information.
⇒ This structure ensures that configuration updates can be tracked and audited.

Testing and Validation

Both pipelines were tested using sample transcript data provided for the assignment.

The demo call transcript was used to verify that the system could correctly extract business information and generate initial configuration files.

The onboarding transcript was used to verify that the system could parse structured information such as email addresses and phone numbers.

The outputs were validated to ensure that:

• Configuration files were generated correctly.
• Updates were merged properly between versions.
• Changelog entries correctly described configuration updates.

Challenges , Solutions and Learnings

During the development of the Clara AI automation pipelines, several practical challenges were encountered while transforming unstructured transcript data into structured agent configurations. Addressing these challenges provided valuable insights into workflow automation, data parsing, and system design.

1. Handling Unstructured Natural Language Transcripts

The demo and onboarding calls were provided as natural language transcripts rather than structured datasets. These transcripts contained conversational sentences, informal phrasing, and varying sentence structures, which made direct data extraction difficult.

Solution:
Regular expressions and keyword-based pattern matching techniques were implemented in the JavaScript code nodes within n8n. This approach allowed the system to detect relevant information such as company names, services, email addresses, and phone numbers even when the wording varied.

Learning:
Processing conversational text requires flexible parsing logic rather than rigid rules. Pattern matching and adaptive extraction techniques are essential when dealing with real-world conversational data.

2. Managing Incomplete Information in Demo Calls

Demo call conversations often contain exploratory discussions where certain business details are not clearly defined. For example, information such as routing rules, integration constraints, or emergency contact details may not be fully specified during the initial conversation.

Solution:
The pipeline was designed to capture missing or uncertain details in a dedicated section called questions_or_unknowns. This ensures that incomplete information is clearly documented and can be resolved during the onboarding stage.

Learning:
Automation systems must be able to handle incomplete data gracefully instead of failing when information is missing.

3. Maintaining Configuration Consistency Between Versions

The system required generating an initial configuration from the demo call and then updating it later using onboarding information. Ensuring compatibility between the version 1 and version 2 configurations while preserving existing data was a challenge.

Solution:
The onboarding pipeline loads the existing configuration file (memo.json) and merges new data with the existing structure rather than replacing it entirely. Additionally, a changelog.json file is generated to track updates made between configuration versions.

Learning:
Version control and change tracking are important when configuration data evolves over time.

4. Designing Flexible Parsing Logic

The onboarding dataset contained timestamped chat messages that could appear in different formats. Relying on hardcoded values would make the system fragile and unable to adapt to new transcripts.

Solution:
Regular expressions were used to dynamically identify patterns such as email addresses, phone numbers, and names. This ensured that the system remained adaptable to variations in transcript formats.

Learning:
Flexible parsing techniques significantly improve the robustness and scalability of automation systems.

5. Structuring Output Data for AI Agent Configuration

The extracted business information needed to be transformed into a structured format compatible with the Clara AI voice agent configuration system.

Solution:
A structured JSON schema was created containing fields such as business hours, emergency definitions, routing rules, and integration constraints. The pipelines automatically populate this schema using the extracted transcript data.

Learning:
Defining a clear data structure is essential for converting raw conversational data into usable system configurations.

6. Designing a Scalable Automation Workflow

The system needed to support future expansion where multiple businesses could be onboarded using the same automation pipelines.

Solution:
A version-based directory structure was implemented:

outputs/accounts/{account_id}/v1
outputs/accounts/{account_id}/v2

This ensures that configurations are organized per account and version.

Learning:
Scalable folder structures and versioning mechanisms help maintain clean data management in automation pipelines.

Conclusion

This project presents an automated system for transforming conversational business transcripts into structured AI agent configurations. By implementing two automation pipelines, the system first generates an initial configuration from demo call transcripts and then refines it using onboarding call information.

The solution leverages n8n workflow automation, JavaScript-based data processing, and JSON configuration files to convert unstructured conversational data into structured outputs suitable for AI voice agent deployment. The use of versioned configuration directories ensures that updates between the demo and onboarding stages are clearly tracked and maintained.

Overall, the developed system demonstrates how workflow automation can streamline the process of configuring AI voice agents, enabling businesses to efficiently convert operational discussions into deployable AI configurations.